

분산 합의에서 리더십의 효과

리더 기반 합의와 리더 없는 합의의 실험적 비교

Sephy Ryu

독립 연구자

대한민국

요약

리더 기반(leader-based) 합의 프로토콜과 리더 없는(leaderless) 합의 프로토콜은 복제본들이 연산을 조정하고 장애로부터 복구하는 방식에서 다릅니다. 본 연구는 Raft와 EPaxos를 각각 리더 기반 및 리더 없는 설계의 대표로 삼아 이러한 차이를 실험적으로 조사합니다. 우리는 복제본 수, 클라이언트 동시성, 읽기/쓰기 구성, 충돌률, 통신 비용에 걸쳐 처리량, 지연 시간, 장애 동작을 평가하며, 여러 구현과 일치된 영속성 구성을 통해 구현 민감도도 함께 살펴봅니다.

결과는 성능 차이가 조건에 크게 의존함을 보여줍니다. 복제본 수와 클라이언트 동시성이 가장 두드러진 차이를 만들며, 읽기/쓰기 구성과 충돌률은 더 약하거나 구현별로 다른 효과를 냅니다. 평가된 Raft 구성은 요청 처리를 선출된 리더에 집중시키고 리더 장애 시 뚜렷한 복구 공백을 보이는 반면, 평가된 EPaxos 구성은 제안 처리를 복제본들에 분산합니다. 그러나 이러한 차이의 크기는 구현과 영속성 선택에 상당히 의존합니다.

일치된 영속성 실험은 기준 구성에서 관찰된 성능 순서를 뒤집을 수 있으며, 통신 실험은 측정된 지연 민감도가 지연을 부과하는 메커니즘과 구현 수준의 메시지 처리에 의존함을 보여줍니다. 따라서 이러한 결과는 Raft와 EPaxos 사이의 보편적인 성능 순위를 지지하지 않습니다. 대신 관찰된 차이는 합의의 구조, 구현, 워크로드, 통신 조건, 장애 메커니즘의 상호작용에서 발생함을 보여줍니다.

본 연구는 결과적으로 합의 프로토콜의 경험적 비교가 구조적 효과와 구현 효과를 구분하고 통제된 운영 조건에 걸쳐 성능을 평가해야 한다고 주장합니다.

Keywords

합의(consensus), Raft, EPaxos, 리더 기반 순서화, 리더 없는 순서화, 벤치마킹

1 서론

합의 프로토콜은 복제본들이 동시 실행과 복제본 장애에도 불구하고 복제 상태 머신이 연산을 일관되게 적용할 수 있도록 조정합니다. 널리 연구된 두 설계 접근 방식은 이러한 조정을 확립하는 방식에서 다릅니다. Raft [6]와 같은 리더 기반 프로토콜은 클라이언트 연산을 선출된 리더를 통해 전달하는 반면, EPaxos [5]와 같은 리더 없는 프로토콜은 복제본들이 독립적으로 연산을 제안하고 일관된 실행에 필요한 의존성을 결정하도록 합니다.

이러한 구조적 차이는 경험적 비교를 동기 부여하지만, 직접적인 프로토콜 수준 순위는 해석하기 어렵습니다. 측정된 처리량과 지연 시간은 복제본 수, 클라이언트 동시성, 워크로드 구성, 충돌률, 통신 조건에 의존합니다. 또한 영속성, 요청 처리, 스케줄링, 전송, 메시지 처리와 같은 구현 메커니즘의 영향을 받습니다. 따라서 리더의 유무는 중요한 구조적 속성이지만, 그 자체로 구체적인 구현의 성능이나 장애 동작을 결정하지는 않습니다. 두 구현 사이의 측정된 차이는 합의 알고리즘 자체에 자동으로 귀속될 수 없습니다.

본 논문에서 우리는 Raft와 EPaxos를 대표적인 리더 기반 및 리더 없는 합의 프로토콜로 실험적으로 비교하며, 통제된 시스템 및 워크로드 조건 변화 하에서 처리량, 지연 시간, 자원 사용률, 장애 동작을 측정합니다.

우리는 다음 연구 질문을 조사합니다:

- **RQ1:** 처리량과 지연 시간은 읽기/쓰기 비율, 클라이언트 동시성, 충돌률, 복제본 수를 포함한 워크로드 및 시스템 조건에 따라 어떻게 변하는가?
- **RQ2:** 평가된 Raft와 EPaxos 구현은 장애 동작과 통신 비용 민감도에서 어떻게 다른가?
- **RQ3:** Raft와 EPaxos 사이의 관찰된 성능 차이는 프로토콜 패밀리 자체보다 구현 선택에 어느 정도 귀속되는가?

우리는 두 프로토콜 패밀리를 공통 컨테이너 환경에서 공유 클라이언트 하네스와 워크로드 생성기를 사용해 실행하는 재현 가능한 실험 프레임워크를 기여합니다. 완전한 벤치마크와 분석 산출물은 <https://github.com/Sephy314/Leader-consensus-comparison>에서 공개됩니다.

우리는 또한 단일 구성 벤치마크가 아닌 민감도 중심 평가를 기여합니다. 실험 스위트는 복제본 수, 클라이언트 동시성, 읽기/쓰기 구성, 충돌률, 장애 모드, 통신 조건을 변화시킵니다.

마지막으로 우리는 두 프로토콜 패밀리의 여러 구현과 일치된 영속성 구성을 사용한 구현 수준 분석을 제공하며, 구현 메커니즘이 측정된 성능 관계를 실질적으로 바꿀 수 있고 일치된 저장 조건에서는 기준 구성에서 관찰된 순서를 뒤집을 수 있음을 보여줍니다.

실험은 로컬 컨테이너 환경에서 수행됩니다. 따라서 보고된 값은 보편적인 생산 성능 추정치가 아니라 지정된 실험 조건에서 평가된 구현의 측정치입니다.

나머지 논문은 관련 연구를 검토하고(Section 2), 실험 방법론을 설명하며(Section 3), 결과를 제시하고 해석하며(Sections 4 및 5), 타당성 위협과 결론을 논의합니다(Sections 6 및 7).

2 배경 및 관련 연구

2.1 리더 기반 및 리더 없는 합의

Raft [6]와 같은 리더 기반 프로토콜은 하나의 복제본을 리더로 지정합니다: 클라이언트 요청은 리더로 전달되고, 리더는 연산을 자신의 로그에 추가하여 팔로워들에게 복제하며, 쿼럼이 이를 확인하면 엔트리가 커밋됩니다. 이는 쓰기 경로를 중앙화하고 리더 장애 시 처리가 재개되기 전에 선거가 필요하게 만듭니다. Paxos [4]는 합의의 기초적인 합의 프레임워크를 확립했지만, 그 추상적인 표현은 시스템 설계자에게 상당한 구현 결정을 남기므로 같은 프로토콜 패밀리의 시스템들도 실행 경로, 저장, 통신 동작에서 크게 다를 수 있습니다.

EPaxos [5]와 같은 리더 없는 프로토콜은 모든 복제본이 연산을 제안하고 그들 사이의 의존성을 결정하도록 합니다. 충돌하지 않는 명령은 단일 전역 순서화 지점 없이 커밋될 수 있지만, 의존

성 추적과 충돌 감지 비용이 듭니다. 복제본들은 여전히 의존성에 동의하고 충돌하는 연산을 호환 가능한 순서로 실행해야 합니다. 하나의 복제본을 잃는 것은 다른 복제본들이 계속 제안하기 전에 대체자를 선출할 필요가 없음을 의미합니다.

2.2 관련 연구

합의 설계에 대한 비교 연구는 조사된 시스템과 워크로드에 따라 다른 결론에 도달했습니다. Howard와 Mortier는 Paxos와 Raft를 비교할 때 프로토콜 패밀리 자체보다 구현과 구성의 선택이 더 중요한 경우가 많다는 것을 발견했습니다 [3]. van Renesse 등은 Paxos와 리더 없는 복제의 상대적 성능이 워크로드, 특히 명령 충돌에 의존함을 보여주었습니다 [9]. Tollman 등은 지리 복제 워크로드에서 EPaxos를 재평가했으며, 원래 벤치마킹 접근 방식이 충돌하는 명령의 성능 영향을 노출하지 않았고 보고된 중앙값보다 훨씬 나쁜 꼬리 지연 시간을 보였다는 것을 발견했습니다 [7]. 이러한 결과는 우리가 여기서 하는 것처럼 통제된 조건에서 각 프로토콜 패밀리의 여러 독립 구현을 평가할 동기를 부여합니다.

우리 연구는 복제본 수, 동시성, 통신 지연, 장애 조건을 변화시키면서 두 Raft 구현(HashiCorp Raft와 etcd Raft)과 두 EPaxos 구현(원본 `efficient/epaxos`와 `nvanbenschoten/epaxos`)을 비교하므로, 결과는 Raft나 EPaxos의 프로토콜 독립적 속성이 아니라 통제된 워크로드 하의 구체적인 구현을 설명합니다.

3 실험 설정

3.1 합의 랩 아키텍처

실험은 평가된 구현을 공통 컨테이너 환경에서 실행하는 재현 가능한 벤치마크 인프라인 Consensus Lab으로 수행됩니다. 각 실험은 하나의 조정 서비스(업스트림 EPaxos 마스터 [5] 또는 랩 제공 Raft 마스터), 복제본당 하나의 컨테이너, 하나의 벤치마크 클라이언트를 포함하는 독립적인 Docker Compose 프로젝트입니다. 모든 컨테이너는 동일한 런타임 이미지와 구성된 CPU 및 메모리 할당을 공유합니다.

두 프로토콜은 업스트림 EPaxos `genericmsmr` 와이어 프로토콜 (`PROPOSE` → `ProposeReplyTS`)에 기반한 동일한 벤치마크 지향 요청 인터페이스를 노출합니다. 두 프로토콜에 단일 클라이언트 바이너리와 워크로드 생성기가 사용되므로 연산 혼합, 키 선택 과정, 동시성 제어, 요청 생성 로직이 공유됩니다. 요청 라우팅은 프로토콜 아키텍처에 따라 다릅니다: Raft 요청은 Raft 마스터가 보고하는 현재 리더로 전달되는 반면, EPaxos 요청은 복제본들에 라운드 로빈으로 분산됩니다. 읽기는 두 프로토콜 모두 합의 경로를 통해 제출됩니다; Raft 특화 로컬 읽기 최적화는 활성화되지 않습니다.

3.2 구현

각 프로토콜은 두 개의 독립 구현으로 대표됩니다. 랩 어댑터는 오케스트레이션, 모니터링, 아래에 설명된 통신 지연 주입만 추가하며 합의 로직을 구현하거나 수정하지 않습니다.

Raft는 `github.com/hashicorp/raft v1.7.3` [2]로 제공되며, 벤치마크 요청을 상태 머신 명령으로 변환하고 `raft.Raft.Apply`를 호출하며 응답을 반환하는 얇은 어댑터(`lab/adapters/raft/`)로 감싸집니다. 리더 선거, 로그 복제, 쿼럼, 커밋, 팀 관리는 라이브러리가 수행합니다. 복제본은 복제본당 Docker 볼륨의 BoltDB 기반 영속 상태를 사용하며, 스냅샷 임계값은 8,192개 엔트리, 후행 로그는 1,024개입니다.

두 번째 Raft 구현은 `go.etcd.io/raft v3.7.0` [1]를 사용하며, 라이브러리의 비동기 저장 쓰기 인터페이스를 따르는 랩 어댑터로 감싸집니다: 전용 `append` 스레드가 로그 엔트리를 단일 쓰기-어헤드 로그 쓰기(한 번의 `fsync`)로 배치하여 HashiCorp Raft의 그물 커밋을 반영합니다. 어댑터는 라이브러리가 요구하는 메시지 크기와 인플라이트 한도를 설정하고, HashiCorp Raft와 일치하도록 `pre-vote`를 활성화하며, 적용된 인덱스 아래로 로그를 압축합니다; 정확한 값은 저장소의 어댑터에 문서화되어 있습니다.

EPaxos는 업스트림 `efficient/epaxos` 서버와 마스터 [5]를 커밋 791b115에 고정하고 `-e -exec -dreply`로 실행합니다. 복제본 상태는 메모리 내에 있습니다. 업스트림 구현은 의존성 집합 크기를 5로 고정합니다; 랩은 프로토콜 단계, 쿼럼, 결정 규칙을 변경하지 않는 문서화된 와이어 형식 확장을 통해 최대 9개 복제본을 지원하도록 이 필드를 확장합니다. 두 번째 EPaxos 구현은 `nvanbenschoten/epaxos` [8]를 커밋 425bd36에 고정하고, 라이브러리의 gRPC 피어 전송을 벤치마크의 클라이언트 프로토콜에 연결하는 랩 어댑터(`lab/adapters/nvbepaxos/`)로 감쌉니다. 두 구현 모두 EPaxos 상태는 메모리 내에 있습니다.

3.2.1 통신 지연 주입. 통신 비용 패밀리는 모든 복제본 간 메시지에 고정된 단방향 지연을 추가하며, 각 구현의 전송 경로에 구현됩니다: HashiCorp Raft의 전송 래퍼는 `AppendEntries`와 `RequestVote`를 전달하기 전에 대기합니다; etcd와 nvb 어댑터의 피어별 전송 고루틴은 쓰기 전에 대기합니다; 원본 EPaxos `genericmsmr` 전송 경로는 각 발신 메시지 전에 대기합니다(랩이 오케스트레이션 외에 해당 바이너리를 수정하는 유일한 곳). 지연은 메시지별로 적용되므로 요청당 더 많은 메시지를 보내거나 전송을 직렬화하는 구현이 더 강하게 영향을 받습니다; etcd 어댑터의 피어별 전송은 임계 경로에서 대기합니다. 따라서 통신 비용 결과는 프로토콜 수준의 지연 민감도 순위가 아니라 추가 지연 하에서 각 구현의 전송 경로 구조의 측정치로 읽어야 합니다.

구현은 자체 네이티브 저장 및 실행 메커니즘을 사용하므로 측정은 고립된 합의 커널이 아니라 완전한 구현을 평가합니다.

3.3 클라이언트 하네스 및 워크로드 모델

벤치마크 클라이언트는 구성 가능한 읽기/쓰기 구성을 가진 1,000개 키의 키스페이스에 걸쳐 요청을 생성합니다. 클라이언트는 폐루프(closed-loop)입니다: 구성된 동시성 단위당 하나의 워커가 이전 요청이 완료된 후에만 다음 요청을 발행하므로 구성된 동시성은 인플라이트 요청의 최대 수입니다. 충돌 실험의 경우 구성된 충돌 비율이 요청이 공유 핫 키 집합에서 선택할 확률을 결정합니다; 실현된 핫 키 비율은 가정하지 않고 생성된 시퀀스에서 측정됩니다. 워크로드 시드는 모든 반복에 대해 기록됩니다. 각 실험은 워밍업 단계 후 측정 단계로 진행됩니다; 워밍업 요청은 성능 측정에 포함되지 않습니다.

3.4 메트릭

각 요청에 대해 클라이언트는 실행 식별자, 프로토콜, 복제본 수, 워크로드 매개변수, 동시성, 연산, 키, 대상 복제본, 시작 및 완료 타임스탬프, 지연 시간, 성공 또는 실패 상태를 기록합니다. 분석 파이프라인은 처리량, 성공률, p50/p95/p99 지연 시간을 도출합니다. 복제본별 자원 텔레메트리는 약 200 ms 해상도로 수집됩니다: `cgroup v2 cpu.stat`의 CPU, `/proc/<pid>/net/dev`의 네트워크 RX/TX, `/proc/<pid>/status`의 상주 메모리. 원시 샘플은 복제본별로 보존됩니다.

표 1: 테스트베드 구성. 텍스트가 달리 말하지 않는 한 이 값은 모든 보고된 실행에 적용됩니다.

매개변수	값
워밍업 / 측정 단계	3 s / 20 s
반복	조건당 10회, 매번 새 Compose 프로젝트
복제본 컨테이너	2.0 CPU, 512 MB, GOMAXPROCS=4
클라이언트 컨테이너	2.0 CPU, 512 MB, GOMAXPROCS=4
조정 서비스	1.0 CPU, 256 MB
요청 시간 초과	2,000 ms
Raft 하트비트 / 선거	1,000 ms / 2,000 ms
저장	Raft 연속, EPaxos 메모리 내
지연 모델	$d = c + U(0, c_j/100)$
비용 c / 지터 j	0, 1, 3, 5, 10 ms / 0, 5, 10, 50, 100 %
격리	반복마다 새 컨테이너, 네트워크, 볼륨

3.5 테스트베드 구성 및 매개변수

Table 1는 모든 보고된 실행의 구성을 나열합니다. 통신 지연 민감도를 조사하기 위해 각 발신 복제본 간 메시지는 $d = c + U(0, c_j/100)$ 만큼 지연될 수 있으며, 여기서 c 는 밀리초 단위의 고정 단방향 비용, j 는 지터 백분율입니다; 이는 물리적 네트워크가 아닌 애플리케이션 수준 메시지 지연을 모델링하며 큐잉, 패킷 손실, 링크 수준 스케줄링을 재현하지 않습니다.

각 반복은 새 컨테이너, 네트워크, 볼륨을 가진 새로운 Docker Compose 프로젝트를 사용합니다. 하네스는 실행 전에 남은 컨테이너, 네트워크, 볼륨, 포트가 없는지 확인하고 실행 후 정리를 검증합니다; 이러한 검사를 통과하지 못한 실행은 조용히 재사용되지 않고 유효하지 않은 시도로 기록됩니다. 구성은 블록 무작위화로 스케줄링됩니다: 각 반복 블록은 시드된 셔플 순서로 각 구성을 정확히 한 번 포함하며, 각 반복은 독립적인 기록된 워크로드 시드를 사용합니다.

3.6 실험 스위트

Table 2는 실험 패밀리를 나열합니다. 처음 일곱 개는 기록된 스위트를 구성하고 마지막 세 개는 나중에 추가되었으며, 각각 자체 결과 하위 트리에서 분석되고 기록된 데이터셋과 결합되지 않습니다 (Sections 4.7–4.9). Workload와 Concurrency 패밀리는 설계상 동시성에서 겹칩니다: Workload 패밀리는 쓰기 비율과 동시성의 상호작용을 측정하고, Concurrency 패밀리는 100% 쓰기에서 동시성 차원을 분리하여 처리량 차이가 가장 큰 256까지 확장합니다. 이 겹침은 실험 간 변동성에 대한 검증도 제공합니다. Election 패밀리는 구성된 목표가 아니라 어댑터의 pre-vote 및 투표 카운터에서 얻은 측정된 실패 선거 횟수를 사용합니다.

장애 주입은 결정적입니다: 리더는 측정 단계의 구성된 오프셋에서 `docker kill`로 선택된 컨테이너를 종료합니다. Raft 리더 장애의 경우 현재 리더는 Raft 마스터에서 얻어 그 컨테이너에 매핑됩니다; 클라이언트는 새 리더에 재연결합니다. EPaxos 복제본 장애의 경우 복제본 0이 종료되고 클라이언트는 남은 복제본으로 이동합니다. 복구 간격은 요청 타임스탬프와 약 200 ms 모니터링 해상도에서 재구성되므로 근사치입니다.

3.7 정확성 검증

성능 실험은 합의 정확성의 공식 검증이 아닙니다. 별도의 하네스 (`runner correctness`)는 각각 고유 키를 쓰는 2,000개의 결정적 요청을 발행하고 읽기 전용 `GetState` RPC가 반환하는 복제본 상태 스냅샷을 비교하므로 두 프로토콜이 동등한 로그 표현을 노출한다고 가정하지 않습니다. 각 구현은 동시성 1, 8,

표 2: 각각 열 번의 독립 반복이 있는 실험 패밀리. 처음 일곱 개는 기록된 스위트를 구성합니다; 마지막 세 개는 나중에 추가되었으며 자체 결과 하위 트리에서 분석되고 그것과 결합되지 않습니다. Section 3의 기본값은 세 복제본, 100 % 쓰기, 동시성 32입니다.

패밀리	변화
Workload	쓰기 비율 0–100 %, 동시성 1–128
Scaling	복제본 3–9
Conflict	구성된 충돌 비율 0–100 %
Concurrency	동시성 1–256
Communication cost	단방향 지연 0–10 ms; 5 ms에서 지터 0–100 %
Election	실패 선거 목표 0–5
Failure	Raft 리더, Raft 팔로워, EPaxos 복제본 종료
Controlled network delay	<code>tc/netem</code> 지연 0–10 ms
Persistence matching	저장 모드, 메모리 대 연속
Conflict validation	충돌 비율 × 핫 키 수

32와 장애 패밀리의 장애 시나리오(영속 로그에서 커밋된 상태를 복구하는 재시작된 Raft 복제본 포함)에서 테스트됩니다. 이는 테스트된 시나리오에 대한 상태 수렴을 확인합니다; 일반적인 선형화 가능성 증명은 아닙니다.

3.8 실험 무결성 및 통계 분석

호스트 수준 텔레메트리(부하 평균, 사용 가능한 메모리)는 모든 실행에 대해 약 200 ms 해상도로 기록되며, 호스트 일시 중지, 스케줄링 공백, CPU 기아, 클럭 이상이 감지되어 기록됩니다. 오염된 것으로 표시된 실행은 집계에서 제외되고 대체되므로 각 구성은 계획된 관측 수를 유지합니다; 대체는 새로운 시도이며 원본은 원시 데이터셋에 남습니다.

모든 집계는 원시 실행별 관측에서 계산됩니다. 소수의 반복이 지연되었기 때문에(Section 4) 처리량과 지연 시간은 반복별 값의 중앙값으로 요약되며, 비모수 부트스트랩(10,000회 재표본)의 95 % 신뢰 구간이 함께 제공됩니다. 프로토콜 차이는 정확한 Mann-Whitney U 검정을, 워크로드 추세 가설은 Kruskal-Wallis 검정을 사용하며, 둘 다 다중성 보정 없이 적용됩니다: 인용된 p -값은 해당 단일 비교의 보정되지 않은 값입니다. 선거 실험은 기술적으로 보고되는 Raft 특화 복구 특성으로 취급되며, 측정된 연관성에 인과적 해석이 부여되지 않습니다.

4 평가

별도로 명시되지 않는 한, 보고된 점 추정치는 열 번의 독립 반복에 대한 중앙값이며, 결과는 세 개의 복제본, 100 % 쓰기, 클라이언트 동시성 32를 사용하고, Section 3의 실험 구성 하에서 평가된 구현을 설명합니다. 모든 그림은 처리된 벤치마크 데이터에서 `lab/report/figures.py`로 생성되며, 반복 간 중앙값을 보여주고 가능한 경우 개별 반복 값을 겹쳐 표시합니다. 실험이 해당 메커니즘을 직접 분리하지 않는 한 차이는 프로토콜 수준의 인과 효과로 해석되어서는 안 됩니다.

4.1 처리량 및 확장성

Figure 1은 복제본 수가 증가함에 따른 측정된 처리량을 보여줍니다. 두 평가된 구현 모두에서 복제본 수가 증가함에 따라 처리량이 감소했습니다. 세 개의 복제본에서 측정된 중앙값은 EPaxos 5,291 req/s, Raft 4,350 req/s였습니다. 다섯 개의 복제본에서는 각각 5,442와 3,148 req/s였고, 아홉 개의 복제본에서는

2,002와 2,112 req/s였습니다(Table 3). 이 실험 패밀리 내 세 복제본 측정치 대비 아홉 복제본 중앙값은 EPaxos에서 62%, Raft에서 51% 낮았습니다. 두 중앙값의 비율은 세 복제본에서 1.22, 다섯에서 1.73, 일곱에서 1.12, 아홉에서 0.95였습니다. 따라서 측정된 처리량 차이는 복제본 수에 따라 달라졌습니다.

우리는 소수의 반복이 지연되었기 때문에 평균이 아닌 중앙값을 보고합니다. 전체 데이터셋(1,302개 실행)에서 8개의 반복이 해당 구성의 중앙값 요청 수의 10% 미만을 완료했습니다: Raft 실행 2개(Raft 실행의 0.3%)와 EPaxos 실행 6개(EPaxos 실행의 1.0%). 8개 중 6개는 실패한 요청 없이 처음부터 끝까지 느렸고, 일곱 및 아홉 복제본의 EPaxos 실행 2개는 지속적인 요청 시간 초과를 보였습니다. Raft 지연 2개는 높은 호스트 부하(건강한 실행의 3.8–4.0 대비 부하 평균 7.0–8.6)와 일치했고, EPaxos 지연 6개는 낮은 부하와 높은 부하 모두에서 발생했습니다. 8개 실행 중 어느 것도 호스트 텔레메트리 오염 규칙에 표시되지 않았으므로 데이터셋에 남아 있습니다. 관찰된 지연 횟수(Raft 실행 680개 중 2개, EPaxos 실행 610개 중 6개)는 유의하게 다르지 않았습니다(Fisher의 정확 검정, $p = 0.16$).

두 번째 이상은 일곱 및 아홉 복제본의 EPaxos에 영향을 미칩니다: 요청 시간 초과입니다. 해당 크기의 EPaxos 실행 20개 중 18개가 2,000ms 클라이언트 시간 초과에서 30–247개 요청이 시간 초과되었고(성공률 98.6–99.9%), 나머지 2개는 모든 요청을 완료했으며 같은 크기의 Raft 실행은 시간 초과가 없었습니다. 각 시간 초과는 성공적인 요청에 사용 가능한 동시성을 줄이면서 클라이언트 워커를 전체 2s 동안 점유합니다. 동시성 N 의 페루프 클라이언트에 대해 회계

$$N \approx X_{ok}W_{ok} + X_{timeout}W_{timeout},$$

여기서 $X_{timeout} = N_{timeout}/W$ 이고 $W_{timeout} = 2s$ 이며, W 는 측정된 단계 창(이 실행들에서 20.0–22.0s)으로, 이 크기의 모든 EPaxos 실행에서 N 의 0.8% 이내로 일치합니다. 성공적인 요청 자체는 p50에서 5.4–5.6 ms로 완료되어 세 복제본 측정과 유사하므로, 처리량 감소는 느린 성공 요청이 아니라 시간 초과까지 워커를 점유하는 요청으로 설명됩니다. 사용 가능한 계층은 시간 초과를 담당하는 특정 실행 경로 메커니즘을 확립하지 않습니다.

동일한 명목 구성(세 복제본, 100% 쓰기, 동시성 32)은 여러 다른 정상 상태 실험 패밀리에서도 독립적으로 발생하여 실행 간 변동성에 대한 지표를 제공합니다. 네 패밀리 중앙값은 Raft에서 4,350, 4,367, 4,482, 4,440 req/s였고 EPaxos에서 5,291, 5,287, 5,175, 5,249 req/s였습니다. EPaxos 중앙값은 약 2%, Raft 중앙값은 약 3% 차이가 났습니다. 따라서 패밀리 간 변동성은 두 구현에서 비슷한 크기였습니다.

확장 패밀리의 경우 구현 간 차이는 세 및 다섯 복제본에서 통계적으로 유의했고(정확한 Mann-Whitney U 검정, 보정되지 않은 $p < 0.001$), 일곱($p = 0.25$) 또는 아홉($p = 0.68$) 복제본에서는 통계적으로 유의한 차이가 감지되지 않았습니다. 중앙값의 95% 부트스트랩 신뢰 구간은 일곱 및 아홉 복제본에서 겹칩니다(Table 3). 따라서 측정된 EPaxos/Raft 처리량 차이는 이 실험에서 더 작은 클러스터 크기에서만 통계적으로 유의했습니다.

Raft 전용 후속 실험은 클라이언트 동시성 128과 256에서(셀당 5회 반복, 40개 실행) 복제본 확장 패널티가 지속됨을 발견했습니다: 아홉 복제본 중앙값은 c128에서 45% 낮았고(11,856에서 6,535 req/s), c256에서 43% 낮았으며(17,325에서 9,949 req/s), c32의 51%와 비교됩니다(Figure 1); r3에서 r9로의 차이는 두 수준 모두에서 유의했습니다($p = 0.008$, 정확한 Mann-Whitney

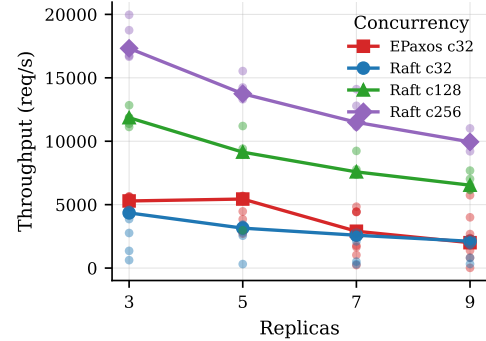


그림 1: 100% 쓰기에서 복제본 수에 따른 처리량. 클라이언트 동시성 32에서의 EPaxos(기록된 비교)와 동시성 32, 128, 256에서의 Raft. 복제본 확장 패널티는 모든 동시성 수준에서 지속됩니다. 점은 개별 반복을, 선은 중앙값을 나타냅니다.

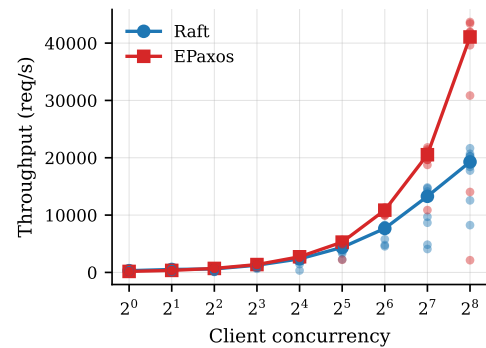


그림 2: 3개 복제본, 100% 쓰기에서 클라이언트 동시성에 따른 중앙값 처리량. 점은 개별 반복을 나타냅니다.

U , 그룹당 $n = 5$). 리더의 CPU 사용률은 동시성에 따라 증가했습니다(세 복제본에서 c32 약 53%에서 c128 82%, c256 103%) 반면 팔로워는 35% 미만을 유지했습니다. 리더는 더 높은 제공 동시성 하에서도 CPU 핫스팟으로 남았으며, 두 코어 할당에 접근했지만 초과하지 않았으므로 이러한 측정은 하드 CPU 용량 한계를 입증하지 않습니다.

Figure 2은 클라이언트 동시성 스위에 걸친 처리량을 보여줍니다. 테스트된 범위에서 두 구현 모두 동시성이 증가함에 따라 처리량이 증가했습니다. 동시성 1에서 측정된 중앙값은 Raft 304 req/s, EPaxos 175 req/s였습니다. 동시성 256에서는 중앙값이 각각 19,278 req/s와 41,069 req/s였습니다. 동시성 256에서 가장 큰 개별 관측은 Raft 21,656 req/s, EPaxos 43,681 req/s였습니다.

동시성 64와 256 사이에서 Raft 중앙값은 7,673에서 19,278 req/s로, EPaxos 중앙값은 10,862에서 41,069 req/s로 증가했습니다. 따라서 두 구현 모두 동시성이 증가함에 따라 처리량을 계속 얻었지만, 증가 크기는 두 구현 간에 달랐습니다.

Figure 3은 쓰기 비율 스위에 걸친 처리량을 보여줍니다. EPaxos 중앙값은 5,165에서 5,461 req/s, Raft 중앙값은 4,219에서 4,474 req/s 범위였습니다. 두 구현 모두 테스트된 쓰기 비율에 걸쳐 단조롭거나 일관된 처리량 추세를 보이지 않았습니다.

표 3: 복제본 수에 따른 처리량과 중앙값(p50) 지연 시간, 100 % 쓰기, 동시성 32. (a) req/s 단위의 처리량과 중앙값의 95 % 부트스트랩 신뢰 구간; 여러 반복이 지연되었으므로 중앙값이 사용되며(Section 4), 평균은 모든 복제본 수의 Raft와 다섯 및 일곱 복제본의 EPaxos에서 더 낮습니다. (b) 열 개의 반복별 p50 값의 중앙값인 밀리초 단위 지연 시간.

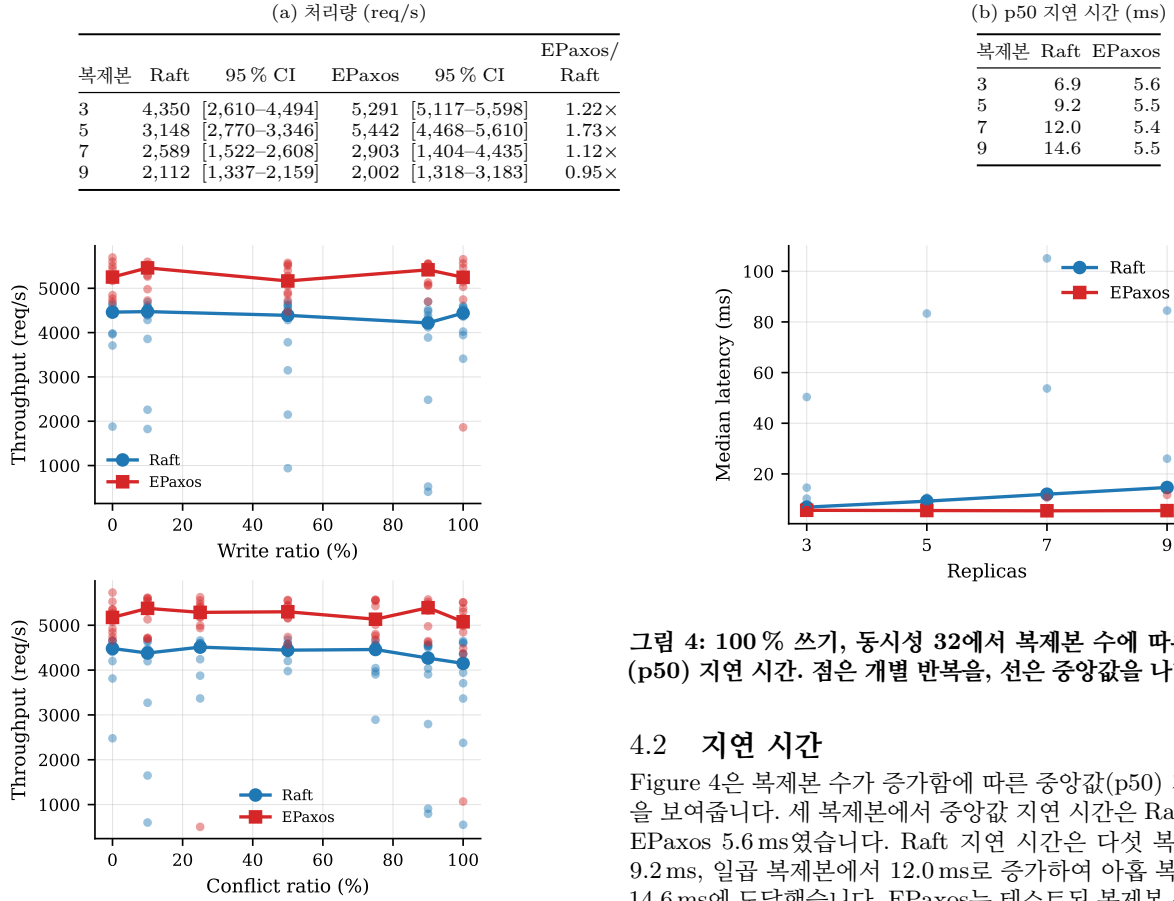


그림 3: 3개 복제본, 동시성 32에서의 처리량. (a) 쓰기 비율 별; (b) 100 % 쓰기에서 구성된 충돌 비율별. 두 스왑 모두 체계적인 처리량 추세를 보이지 않습니다. 점은 개별 반복을, 선은 중앙값을 나타냅니다.

가장 높은 EPaxos 중앙값은 10 % 쓰기에서 발생했으며 100 % 쓰기 측정치보다 약 4 % 높았고, 가장 높은 Raft 중앙값도 10 % 쓰기에서 발생했습니다.

쓰기 비율에 따른 변동은 통계적으로 유의하지 않았습니다. 다섯 개의 쓰기 비율 그룹에 대한 Kruskal–Wallis 검정은 Raft에서 $H = 1.68$, EPaxos에서 $H = 1.75$ 로 둘 다 χ^2_4 임계값 9.49보다 낮았습니다. 따라서 측정된 조건에서 쓰기 비율과 연관된 체계적인 처리량 효과의 증거를 제공하지 않습니다.

Figure 3은 구성된 충돌 비율 스왑에 걸친 처리량을 보여줍니다. 중앙값 처리량은 Raft에서 4,149에서 4,512 req/s, EPaxos에서 5,078에서 5,391 req/s 범위였습니다. 일곱 개의 충돌 비율 그룹에 대한 Kruskal–Wallis 검정은 Raft에서 $H = 4.44$, EPaxos에서 $H = 3.13$ 로 둘 다 χ^2_6 임계값 12.59보다 낮았습니다. 두 구현 모두에서 구성된 충돌 비율과 단조로운 처리량 관계가 관찰되지 않았습니다.

그림 4: 100 % 쓰기, 동시성 32에서 복제본 수에 따른 중앙값(p50) 지연 시간. 점은 개별 반복을, 선은 중앙값을 나타냅니다.

4.2 지연 시간

Figure 4은 복제본 수가 증가함에 따른 중앙값(p50) 지연 시간을 보여줍니다. 세 복제본에서 중앙값 지연 시간은 Raft 6.9 ms, EPaxos 5.6 ms였습니다. Raft 지연 시간은 다섯 복제본에서 9.2 ms, 일곱 복제본에서 12.0 ms로 증가하여 아홉 복제본에서 14.6 ms에 도달했습니다. EPaxos는 테스트된 복제본 수에 걸쳐 5.4–5.6 ms의 비교적 좁은 범위를 유지했습니다(Table 3b).

따라서 Raft p50은 복제본 수에 따라 단조롭게 증가한 반면 EPaxos p50은 5.4–5.6 ms 범위에 머물렀습니다.

p50 값은 꼬리와 함께 읽어야 합니다: p99는 세 복제본에서 Raft 21.4 ms, EPaxos 23.8 ms였으며, 일곱에서 아홉 복제본에서 Raft는 26–29 ms로 증가한 반면 EPaxos는 14–16 ms에 머물렀습니다. 반복별 p50 값의 중앙값은 앞서 설명한 지연된 반복의 영향을 줄입니다.

Figure 5은 클라이언트 동시성에 걸친 중앙값 지연 시간을 보여줍니다. 동시성 1에서 중앙값 지연 시간은 Raft 2.8 ms, EPaxos 5.5 ms였습니다. 동시성 32에서는 각각 6.9 ms와 5.6 ms였습니다. 동시성 256에서는 12.1 ms와 5.6 ms였습니다.

따라서 상대적 지연 시간 순서는 동시성에 따라 바뀌었습니다: Raft는 동시성 1에서 더 낮은 중앙값을 가졌고, EPaxos는 동시성 32 이상에서 더 낮은 중앙값을 가졌습니다.

처리량과 마찬가지로 동일한 명목 구성(세 복제본, 동시성 32)이 여러 실험 패밀리에서 측정됩니다. 중앙값 기준으로 p50은 패밀리 간에 일관되었습니다: Raft는 확장 패밀리에서 6.9 ms, 동시성 패밀리에서 6.9 ms를 측정했고, EPaxos는 둘 다에서 5.6 ms를 측정했습니다.

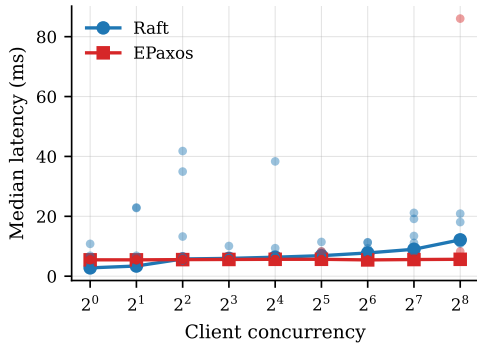


그림 5: 3개 복제본, 100 % 쓰기에서 클라이언트 동시성에 따른 중앙값 (p50) 지연 시간. 점은 개별 반복을, 선은 중앙값을 나타냅니다.

페루프 클라이언트의 경우 Little의 법칙은 평균 지연 시간을 동시성과 처리량에 $W = N/X$ 로 관련짓습니다. 보고된 p50은 평균이 아닌 중앙값이므로 이 값과 같을 필요는 없습니다. 동시성 256에서 Raft p50 12.1 ms는 $N/X(256/19,278 \approx 13.3 \text{ ms})$ 에 가까웠고, EPaxos p50 5.6 ms는 $N/X(256/41,069 \approx 6.2 \text{ ms})$ 보다 낮았습니다. 이 차이는 중앙값이 평균보다 낮은 응답 시간 분포와 양립합니다. 우리는 처리된 데이터셋에서 p50, p95, p99를 보고하고 여기서는 중심 경향 요약으로 p50을 사용합니다.

일곱 및 아홉 복제본에서 EPaxos p50 5.4–5.5 ms는 $N/X(32/2,002 \sim 2,903 \approx 11.0 \sim 16.0 \text{ ms})$ 보다 상당히 낮았습니다. Section 4에서 설명한 대로 이 차이는 해당 크기의 요청 시간 초과로 설명됩니다: 각 시간 초과는 워커를 전체 2s 동안 점유하며, 시간 초과 항목을 포함한 페루프 동시성 회계는 모든 EPaxos 실행과 일치합니다. 같은 크기의 Raft 실행은 시간 초과가 없었고 측정된 N/X 값은 보고된 p50 지연 시간에 더 가까웠습니다.

4.3 자원 사용률

Figure 6은 복제본별 자원 측정을 요약합니다. 평가된 실행에 걸쳐 평균 복제본별 CPU 사용률은 Raft 36.5%, EPaxos 35.4%였습니다. 평균 최대 복제본별 RSS는 Raft 28.3 MB, EPaxos 46.4 MB였습니다. 평균 복제본별 네트워크 속도는 Raft 451 kB/s RX, 454 kB/s TX였고, EPaxos 410 kB/s RX, 394 kB/s TX였습니다.

Figure 7은 아홉 복제본 확장 실험에서 CPU 사용률의 분포를 조사합니다. Raft 복제본별 평균은 16.3%에서 29.4% 범위였습니다. 주어진 반복 동안 리더 역할을 한 복제본은 팔로워보다 상당히 더 많은 CPU를 소비했습니다. 각 실행의 Raft 로그에서 리더를 가져와, 평균 사용률의 리더/팔로워 비율은 세 복제본에서 2.2, 다섯에서 2.8, 일곱에서 3.3, 아홉에서 3.8이었으며(열 반복의 중앙값 사용), 클러스터가 커짐에 따라 리더의 CPU 비용이 각 팔로워 대비 증가했습니다. (리더의 클러스터 총 CPU 점유율은 복제본이 추가됨에 따라 감소합니다; 증가하는 것은 팔로워당 비율입니다.)

이는 평가된 HashiCorp Raft 구현의 요청 경로와 일치합니다: 리더는 벤치마크 클라이언트를 서비스하고 모든 명령을 적용하며 자체 로그를 영속화하고 모든 피어에게 브로드캐스트하는 반면, 팔로워는 수신, 영속화, 적용만 합니다. 한 세 복제본 실행의 CPU 프로파일은 두 역할 모두에서 동일한 작업 형태를 보여주며, 샘

표 4: 3개 복제본, 100 % 쓰기, 동시성 32에서의 장애 동작. 공백은 장애 주입 후 성공적인 쓰기 완료가 없는 가장 긴 간격입니다.

장애	프로토콜	공백 (s)	실패 요청
리더 종료	Raft	1.81–4.74	2,169–4,133
팔로워 종료	Raft	0.02–0.04	0
복제본 종료	EPaxos	0.02–0.09	10–106

플의 약 2/3가 영속성 경로의 시스템 호출과 락 대기에 사용되고, 리더는 팔로워와 주로 수행하는 작업의 양에서 다릅니다. 계측은 영속성, 적용, 전송 기여를 분리하지 않지만 방향은 명확합니다: 평가된 구현에서 선출된 리더는 CPU 핫스팟이었고, 그 상대적 비용은 서비스하는 피어 수에 따라 증가했습니다.

4.4 장애 복구 동작

장애 결과는 장애 유형에 따라 달랐습니다(Table 4, Figure 8). Raft 리더를 종료하면 쓰기 가용성 공백 1.81–4.74s와 2,169–4,133개의 실패한 요청이 발생했습니다. Raft 팔로워를 종료하면 실패한 요청 없이 0.02–0.04s의 공백이 발생했습니다. EPaxos 복제본을 종료하면 10–106개의 실패한 요청과 함께 0.02–0.09s의 공백이 발생했습니다.

모든 실행은 실험 하네스에 따라 성공적으로 완료되었습니다; 보고된 실패한 요청은 전환 중에 발행된 요청에 해당하며 실행 수준 실패와 구별됩니다.

Raft 리더 종료 후 더 긴 가용성 공백은 구성된 Raft 선거 및 요청 재시도 동작과 일치했습니다. 대조적으로 테스트된 팔로워 장애 사례는 Raft 리더의 교체를 요구하지 않았고, EPaxos 복제본 장애 사례는 남은 복제본으로 계속되었습니다. 이러한 결과는 테스트된 장애 시나리오를 설명하며 모든 리더 기반 또는 리더 없는 구현의 대표로 해석되어서는 안 됩니다.

선거 실험은 반복된 선거 중단 하에서 Raft 특화 복구의 더 통제된 특성을 제공합니다. 측정된 실패 선거 수는 60개 실행에 걸쳐 0에서 13까지 범위였습니다. 가용성 공백은 측정된 선거 수와 강하게 연관되었지만 ($r = 0.90, p < 10^{-4}$), 이 관계는 부분적으로 기계적입니다. 왜냐하면 격리 간격 자체가 구성된 중단 기간에 따라 증가하기 때문입니다.

격리 간격과 독립적으로 복구를 조사하기 위해 분석은 격리 창 끝에서 첫 성공적인 요청까지의 시간도 측정합니다. 이 격리 후 복구 간격은 더 가변적이었고 측정된 실패 선거 수와 더 약한 양의 연관성을 보였습니다 ($r = 0.41, p = 0.001, n = 60$). 이 결과는 인과 관계의 증거가 아닌 관찰된 연관성으로 보고됩니다.

4.5 정확성 검증

Section 3에서 설명한 정확성 하네스는 테스트된 워크로드와 장애 시나리오 하에서 상태 수렴을 확인합니다. 18개 테스트 모두 통과했습니다 (Table 5): 모든 응답된 요청이 모든 복제본에서 정확히 한 번 적용되었고, 생존 복제본이 최종 상태에 동의했으며, 재시작된 Raft 복제본이 영속 로그에서 커밋된 상태를 복구했고, 각 장애 후 발행된 요청이 정상적으로 처리되었습니다. 정확성 워크로드가 각 요청에 고유 키를 할당하므로 이는 주로 상태 수렴 검사이며 연산 순서 의미론의 독립적인 검증이 아닙니다.

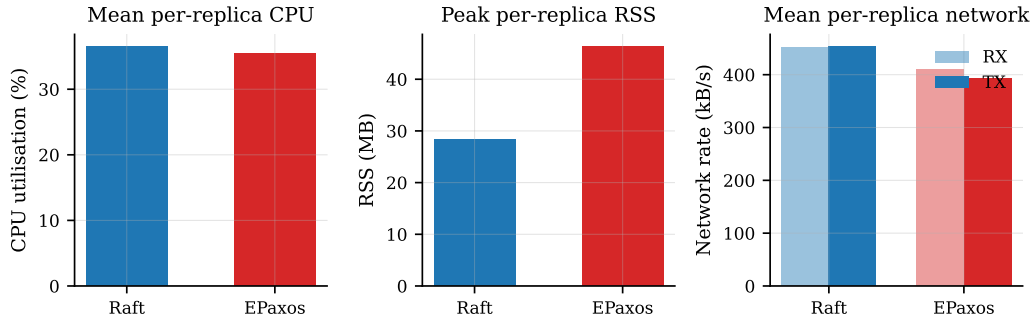


그림 6: 평가된 실행에 걸쳐 집계된 평균 복제본별 CPU 사용률, 최대 복제본별 RSS, 평균 복제본별 네트워크 속도.

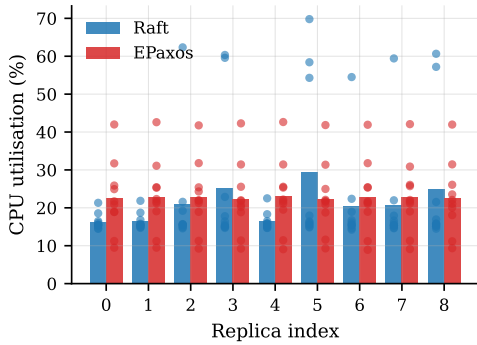


그림 7: 9개 복제본 확장 실행에서 복제본별 CPU 사용률. 막대는 반복 간 평균을, 점은 개별 반복을 나타냅니다.

표 5: 정확성 검증 결과: 3개 복제본, 100 % 쓰기에서 테스트당 2,000개의 결정적 요청. 모든 응답된 요청이 모든 복제본에서 정확히 한 번 적용되었고, 생존 복제본이 최종 상태에 동의했으며, 각 장애 후 발행된 요청이 정상적으로 처리되었습니다.

구현	순차	c=8	c=32	장애
Raft (HashiCorp)	PASS	PASS	PASS	팔로워/리더
Raft (etcd)	PASS	PASS	PASS	팔로워/리더
EPaxos (original)	PASS	PASS	PASS	복제본
EPaxos (nvb)	PASS	PASS	PASS	복제본

4.6 통신 비용

통신 비용 패밀리는 추가 애플리케이션 수준 단방향 메시지 지연에 대한 민감도를 평가하며, 물리적 네트워크의 직접 시뮬레이션이 아닌 통제된 통신 조건 실험으로 해석되어야 합니다.

Figure 10a는 네 구현에 대해 지터가 없는 각 지연 수준에서의 평균 처리량을 보여줍니다. 구현들은 주입된 지연에 대해 상당히 다른 민감도를 보였습니다. HashiCorp Raft는 0 지연의 4,502 req/s에서 10 ms의 1,280 req/s로 감소하여 72 % 감소했습니다. nvb EPaxos는 4,346에서 833 req/s로 81 % 감소했고, 원본 EPaxos 구현은 5,527에서 221 req/s로 96 % 감소했습니다. etcd Raft 구현은 다른 응답을 보였습니다: 처리량이 0 지연의

6,420 req/s에서 1 ms의 375 req/s, 10 ms의 45 req/s로 감소했습니다.

따라서 측정된 순서는 추가 통신 지연 하에서 구현 간에 달랐습니다. 평균의 95 % 신뢰 구간은 모든 0이 아닌 지연 수준에서 평가된 구현 사이에 분리되었습니다.

중앙값 지연 시간은 네 구현 모두에서 주입된 지연에 따라 증가했습니다. 10 ms에서 측정된 p50 값은 HashiCorp Raft 22.6 ms, nvb EPaxos 36.2 ms, 원본 EPaxos 125.9 ms, etcd Raft 703.4 ms였습니다. 이 조건에서 지연 시간에 대해 관찰된 구현 순서는 처리량 순서와 일치했습니다.

지터 스위프는 고정 지연의 해당 증가보다 더 작은 변화를 만들었습니다. 5 ms 기본 지연에서 지터를 0에서 100 %로 증가시키면 처리량이 HashiCorp Raft에서 2,316에서 1,760 req/s(24 %), nvb EPaxos에서 1,432에서 1,082 req/s(24 %), 원본 EPaxos에서 479에서 305 req/s(36 %), etcd Raft에서 87에서 59 req/s(32 %)로 감소했습니다.

구현된 지터 모델에서 100 % 지터는 평균 주입 지연도 5에서 7.5 ms로 올리므로 실험은 지연 분산과 평균 비용의 증가를 분리할 수 없습니다.

4.7 영속성 일치

기록된 기준선은 로그를 저장소에 영속화하는 Raft 구현과 복제본 상태를 메모리에 유지하는 EPaxos 구현을 비교합니다(Section 6). 영속성 일치 패밀리는 워크로드, 복제본 수, 동시성, 네트워크를 고정하고 구현 자체가 제공하는 모드만 사용하여 저장 모드만 변화시킵니다. 자체 결과 하위 트리에서 분석되며 기록된 데이터셋과 결합되지 않습니다; 147개 실험 중 140개가 포함되었고, Section 4.9 끝에 설명된 대체 후 모든 조건이 계획된 열번의 반복을 유지합니다.

Table 6와 Figure 11는 세 및 다섯 복제본에서의 처리량을 보여줍니다. 기록된 기준선을 재현하는 두 모드는 그것과 일치합니다: 세 복제본에서 영속 HashiCorp Raft 중앙값 4,425 req/s는 기록된 4,350 req/s보다 1.7 % 높고, 메모리 내 원본 EPaxos 중앙값 5,585 req/s는 기록된 5,291 req/s보다 5.6 % 높으며, 위에서 보고된 2-3 % 패밀리는 분포의 상한입니다.

저장 모드를 일치시키면 두 복제본 수 모두에서 비교 방향이 바뀝니다. 두 구현 모두 메모리 내일 때 HashiCorp Raft는 원본 EPaxos의 5,585 req/s에 대해 29,660 req/s를 측정했습니다(5.3×); 둘 다 영속일 때 4,425 대 2,005(2.2×). 두 번째 쌍에서 etcd Raft는 세 복제본 메모리 내에서 nvb EPaxos의 4,226 req/s에 대해 12,884 req/s(3.0×), 다섯 복제본에서 1,600

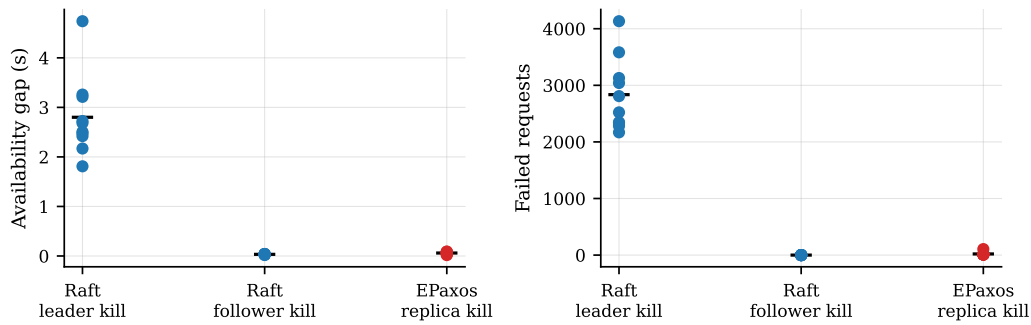


그림 8: 3개 복제본, 100 % 쓰기, 동시성 32에서 장애 유형별 쓰기 가용성 공백과 실패한 요청. 점은 개별 반복을, 막대는 평균을 나타냅니다.

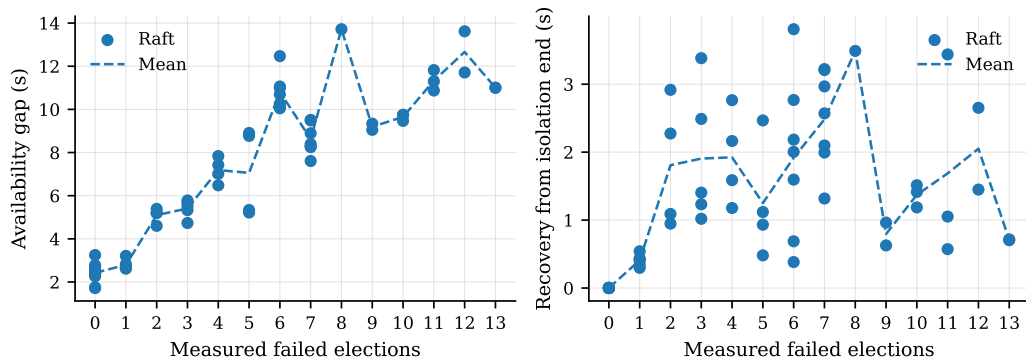


그림 9: 3개 복제본, 100 % 쓰기, 동시성 32에서 Raft의 선거 장애 복구. 왼쪽: 측정된 실패 선거 수에 따른 쓰기 가용성 공백. 오른쪽: 격리 창 끝에서 첫 성공적인 요청까지의 복구 시간. 점은 개별 반복을, 점선은 값별 평균을 나타냅니다.

에 대해 8,437(5.3×)을 측정했습니다. 따라서 이 패밀리의 모든 일치된 비교는 Raft 구현을 EPaxos 구현 위에 두며, 각각 유의합니다(정확한 Mann-Whitney U 검정, $p < 0.001$). 두 번째 쌍의 연속 셀에는 EPaxos 대용이 없습니다: nvb EPaxos 라이브리는 메모리 내 저장만 제공하며 공백을 채우기 위해 연속성 메커니즘이 추가되지 않았으므로 그림은 해당 막대를 Raft만 표시합니다.

저장 모드가 역전을 설명합니다. 동일 구현의 메모리 내 중앙값 대비 연속 저장은 세 복제본에서 HashiCorp Raft 15%, etcd Raft 48%, 원본 EPaxos 36%를 유지했고, 다섯 복제본에서 13%, 69%, 33%였습니다(각 비교 $p < 0.001$). 따라서 기록된 비교는 가장 큰 연속성 비용을 가진 구현(HashiCorp Raft, 로그가 연속화될 때 메모리 내 처리량의 85% 손실)을 평가된 모드가 메모리 내인 구현(연속화될 때 64% 손실하는 원본 EPaxos)과 비교했습니다. 기록된 EPaxos/Raft 비율 1.22는 그 짝과 일치하므로 두 프로토콜 패밀리의 속성으로 읽어서는 안 됩니다. 이 패밀리는 두 복제본 수에서 하나의 워크로드를 측정하므로 기록된 순서가 저장 모드에 의존함을 확립합니다; 연속성 일치 배포가 다른 복제본 수나 쓰기 비율에서 두 패밀리를 같은 방식으로 순서화할 것이라는 것은 확립하지 않습니다.

p50 순서는 처리량 순서를 따릅니다. 세 및 다섯 복제본에서 연속 HashiCorp Raft는 메모리 내 0.8 및 0.9ms에 대해 6.8 및 9.3ms를 측정했습니다; 연속 etcd Raft는 메모리 내 1.7 및

표 6: 3개 및 5개 복제본, 100 % 쓰기, 동시성 32에서의 연속성 일치. 셀은 같은 쌍의 평가된 EPaxos 구현에 대한 평균된 Raft 구현의 중앙값 처리량(req/s)을 주며, 괄호 안에 Raft/EPaxos 비율이 있습니다. 모든 셀은 열 개의 포함된 반복의 중앙값입니다. 모드는 구현 자체의 것입니다: HashiCorp Raft BoltStore 대 NewInmemStore, etcd Raft 쓰기-어헤드 로그 대 -no-wal, efficient/epaxos 업스트림 -durable 대 메모리 내 기본값. nvanbenshoten/epaxos는 연속 모드를 제공하지 않으므로 해당 셀은 공백이며 추가 메커니즘으로 채워지지 않습니다.

연속성	복제본	HashiCorp Raft / 원본 EPaxos	etcd Raft / nvb EPaxos
연속	3	4,425 / 2,005 [2.2×]	공백
연속	5	3,294 / 1,838 [1.8×]	공백
메모리	3	29,660 / 5,585 [5.3×]	12,884 / 4,226 [3.0×]
메모리	5	24,573 / 5,613 [4.4×]	8,437 / 1,600 [5.3×]

2.9ms에 대해 4.9 및 5.1 ms; 연속 원본 EPaxos는 메모리 내 5.7 및 5.5ms에 대해 14.5 및 15.6 ms; 메모리 내 nvb EPaxos는 7.1 및 18.6 ms였습니다.

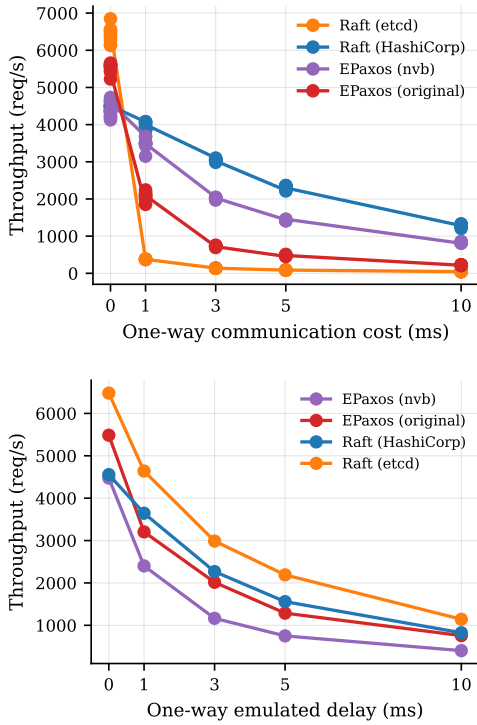


그림 10: 3개 복제본, 100 % 쓰기, 동시성 32에서 네 평가 구현에 대한 추가 단방향 지연에 따른 처리량. (a) 각 구현의 전송 경로 내부에 주입된 지연으로, 구현들이 급격히 갈라집니다. (b) 구현을 수정하지 않고 tc/netem으로 복제본 간 트래픽에 적용된 동일한 명목 지연으로, 분포가 훨씬 좁습니다.

4.8 통제된 네트워크 지연

위에서 설명한 통신 비용 패밀리는 각 구현의 전송 경로 내부에 지연을 주입합니다. 통제된 네트워크 지연 패밀리는 구현 외부에서 동일한 명목 조건을 적용합니다: 리더가 각 복제본의 네트워크 네임스페이스 내부에 tc/netem qdisc를 설치하고 다른 복제본으로 향하는 트래픽만 지연시키므로 어댑터가 대기하지 않고 합의 소스가 수정되지 않습니다. 0ms 지점은 qdisc 없이 기록된 패밀리 자체의 기준선이며 200개 실행 모두 포함되었습니다.

중앙값 처리량은 네 구현 모두에서 적용된 지연에 따라 단조롭게 감소했습니다(Figure 10b); 다섯 지연 수준에 대한 Kruskal-Wallis 검정은 각 구현에 대해 $\chi^2_4 = 9.49$ 대비 $H = 47.06$ 을 주며, 열 반복의 관찰된 범위는 어느 구현에서도 인접 지연 수준 사이에 겹치지 않습니다. 10ms에서 구현들은 0ms 중앙값의 18.1%(HashiCorp Raft), 17.6%(etcd Raft), 13.8%(원본 EPaxos), 9.0%(nvb EPaxos)를 유지했습니다. 10ms에서 p50 지연 시간은 각각 36.9, 25.7, 44.6, 83.0ms였으며 0ms에서는 6.6, 4.7, 5.7, 6.8ms였습니다.

통제된 조건은 애플리케이션 수준 패밀리가 측정한 구현 간 차이를 압축합니다: 동일한 명목 10ms에서 유지된 0ms 처리량의 비율은 네 구현에 걸쳐 9.0–18.1%로 2배 범위인 반면, 애플리케이션 수준 패밀리는 0.7–28.4%로 40배 범위였습니다. 따라서 애플리케이션 수준 패밀리가 측정한 etcd Raft의 붕괴는 추가 지연 자체가 아니라 주입 메커니즘, 즉 어댑터 전송 경로의

메시지별 대기에 기인하며, Section 6가 예상한 대로입니다. 순서도 바뀝니다: 애플리케이션 수준 지연에서 가장 많이 유지한 두 구현(HashiCorp Raft 28.4%, nvb EPaxos 19.2%)은 OS 수준 에뮬레이션에서 더 적게 유지합니다(18.1% 및 9.0%). 이는 에뮬레이션이 단일 전송이 아닌 양방향으로 각 복제본 간 교환을 지연시키는 것과 일치합니다. 두 메커니즘은 상호 교환할 수 없으며, 통제된 패밀리가 여기서 지연 민감도의 증거로 사용됩니다.

4.9 충돌 검증

기록된 충돌 비율 스웬은 워크로드가 핫 키를 대상으로 하는 빈도를 구성하지만, 구성된 비율 자체는 실현된 접근 패턴이나 생성된 프로토콜 수준 경합을 확립하지 않습니다. 충돌 검증 패밀리는 구성된 핫 키 비율과 그 비율이 대상으로 하는 고유 핫 키 수를 모두 변화시키면서 스웬을 재실행하고 요청별 핫 플래그에서 실현된 비율을 기록합니다; 492개 실행 중 480개가 포함되었고, 12개는 표시된 시도 3개와 대체된 전임자 9개로 제외되어 48개 조건 모두 열 번의 반복을 포함합니다.

Figure 12(a)는 워크로드가 구성한 것을 전달했음을 보여줍니다. 단일 핫 키로 최상위 키의 요청 점유율은 구성된 비율과 같았고(10, 25, 50, 75, 100%에서 9.9, 24.9, 49.7, 74.8, 100.0%), 여덟 개의 핫 키로는 설계 의도대로 그 1/8이었습니다. 100%에서 워크로드는 냉기를 전혀 포함하지 않아 실행당 하나 또는 여덟 개의 고유 키만 관찰되므로 해당 지점은 스웬의 나머지와 같은 연속체에 있지 않으며 아래에 별도로 보고됩니다.

처리량은 구성된 비율과 체계적인 관계를 보이지 않았습니다. HashiCorp Raft의 중앙값은 두 키 집합 크기에서 4,354에서 4,541 req/s, etcd Raft는 5,848에서 6,301 req/s, 원본 EPaxos는 5,448에서 5,517 req/s 범위였습니다. 이 세 구현의 여섯 조합에 대한 0–75% 지점의 Kruskal-Wallis 검정은 $\chi^2_4 = 9.49$ 대비 $H \leq 6.73$ 을 주며, p50 중앙값은 각각 6.7–6.9ms, 4.8–5.1ms, 5.66–5.68ms에 머물렀습니다. 이는 이 구현들에 대한 기록된 귀무 결과를 재현하며 구성된 비율이 의도된 접근 패턴을 만들지 못해서가 아님을 보여줍니다. 코리도 0–75%에서 같은 방식으로 동작합니다: p99 중앙값은 etcd Raft에서 10.4–11.4ms, HashiCorp Raft에서 14.8–17.3ms, 원본 EPaxos에서 13.3–17.0ms에 머무는 반면, nvb EPaxos는 중앙값과 함께 16.6에서 36.1ms로 이동합니다(0% 대비 75%에서 2.04× 및 2.12×).

nvb EPaxos는 예외입니다. 중앙값 처리량은 0–75%에 걸쳐 구성된 비율에 따라 단조롭게 감소했으며, 하나의 핫 키로 4,123에서 2,129 req/s, 여덟 개로 4,238에서 2,324 req/s($H = 42.28$ 및 $H = 41.84$), p50은 7.3에서 13.9ms, 7.1에서 12.4ms로 상승했습니다. 100%에서 두 중앙값 모두 0% 값 위로 상승하여 6,103 req/s와 6,631 req/s, p50 5.1 및 4.5ms에 도달했습니다. 역전은 해당 지점의 퇴화된 키스페이스와 일치합니다; 계측은 그 메커니즘을 확립하지 않으며 이 구현은 더 조사할 프로토콜 카운터를 노출하지 않습니다. 따라서 이 패밀리에서 충돌 민감도는 구현별입니다: 네 구현 중 세 개는 키스페이스가 여전히 냉기를 포함하는 범위에서 구성된 비율에 대한 처리량 응답을 보이지 않는 반면, 네 번째는 단조롭게 감소합니다.

Figure 12(b)는 사용 가능한 유일한 프로토콜 수준 지표를 보고합니다: 원본 EPaxos 구현의 슬로우 패스 점유율은 구성된 비율이 증가함에 따라 감소하며, 하나의 핫 키로 97.8에서 43.8%, 여덟 개로 97.9에서 72.5%입니다. conflicted 카운터도 함께 움직이지만, 이 구현에서 그것은 키 충돌이 아니라 인스턴스의 시퀀스 번호와 의존성에 대해 PreAccept 응답이 불일치하는 복제본 수를 셉니다. 두 카운터 모두 클라이언트 연결이 수락

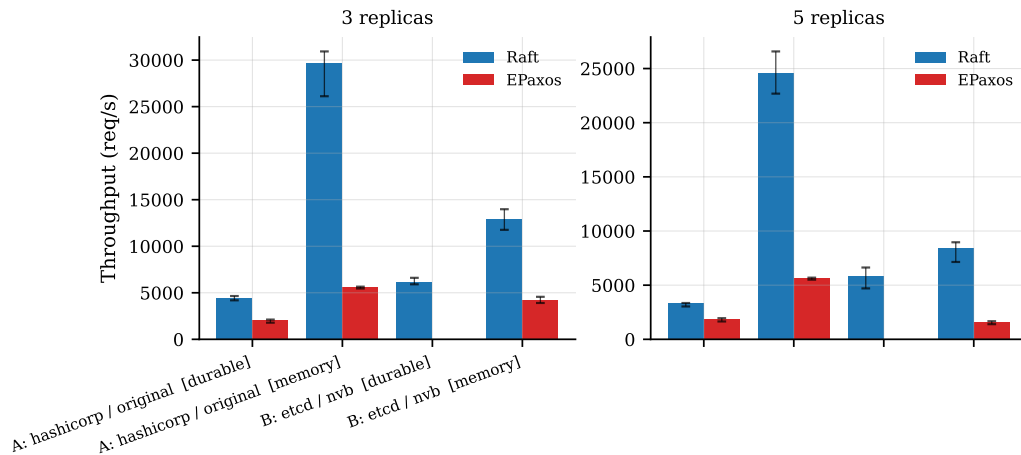


그림 11: 100 % 쓰기, 동시성 32에서 두 구현 쌍에 대한 저장 모드별 중앙값 처리량(req/s). 막대는 포함된 반복의 중앙값을, 오차 막대는 관찰된 범위를 나타냅니다. 쌍 A는 HashiCorp Raft 대 원본 EPaxos, 쌍 B는 etcd Raft 대 nvb EPaxos입니다. 쌍 B의 연속 셀은 nvb EPaxos 라이브러리가 연속 모드를 제공하지 않으므로 Raft 막대만 포함합니다.

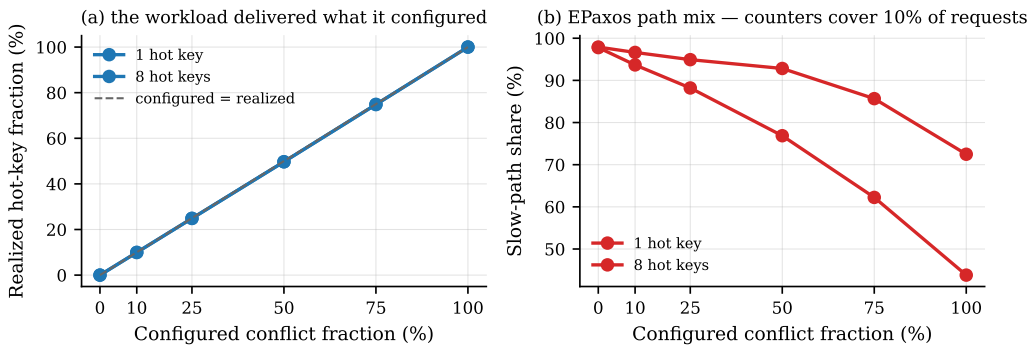


그림 12: 3개 복제본, 100 % 쓰기, 동시성 32에서의 충돌 검증. 왼쪽: 요청별 핫 플래그가 정확하게 만드는 구성된 비율에 대한 실현된 핫 키 비율. 오른쪽: 경로 카운터를 노출하는 유일한 구현의 EPaxos 슬로우 패스 점유율; 카운터는 서버가 클라이언트 연결을 수락할 때마다 재설정되므로 실행의 성공적인 요청 약 9.5%를 덮습니다.

될 때마다 재설정되므로 측정 단계가 아닌 성공적인 요청의 약 9.5%를 덮습니다; 그 덮개는 이 패밀리의 조건들 사이에서 거의 일정하므로 점유율을 비교 가능하게 하지만 창 내 지표로 남게 하며 전체 조사는 아닙니다. 그 한계를 감안하면 방향은 더 높은 구성된 충돌 비율이 더 많은 슬로우 패스 실행을 강제한다는 기대와 반대입니다. 더 뜨거운 키의 인스턴스가 해당 키의 의존성 체인에 직렬화되어 복제본이 더 자주 동의하는 것과 일치합니다; 사용 가능한 계층은 이 해석을 다른 해석과 분리하지 않으며 nvb EPaxos에는 카운터가 없습니다.

세 패밀리에 두 가지 대체 규칙이 적용되었으며, 둘 다 원래 시도를 원시 데이터셋에 유지합니다. 실패했거나 호스트 텔레메트리가 오염된 것으로 표시한 시도는 새 시도로 재실행되었습니다. 표시되지 않은 채 이상한 반복은 그 규칙에 보이지 않으므로 두 번째 규칙이 사전에 고정되어 모든 조건에 균일하게 적용되었습니다: 자체 구성의 중앙값의 0.7배 미만을 측정하는 포함된 반복은 재실행되며 보고서는 값에 관계없이 대체를 유지합니다. 열한 개의 반복이 두 번째 규칙에 해당했고 일곱 개의 시도가

첫 번째 규칙으로 대체되었습니다. 세 패밀리의 포함된 반복 중 이제 자체 조건 중앙값의 0.7배 미만을 측정하는 것은 없고, 어떤 반복의 p50도 자체 조건의 1.5배를 초과하지 않으며, 모든 조건이 계획된 열 개의 관측을 가집니다; 해당 반복을 대체해도 이 섹션에서 보고된 중앙값은 3% 이상 움직이지 않았습니다.

4.10 발견 요약

측정은 평가된 조건 하에서 다음 관찰을 지원합니다:

- 두 구현 모두에서 복제본 수가 증가함에 따라 처리량이 감소했지만, EPaxos/Raft 비율은 클러스터 크기에 따라 바뀌었고 차이는 세 및 다섯 복제본에서만 통계적으로 감지 가능했습니다.
- 1,302개 반복 중 8개가 지연되어(두 Raft, 여섯 EPaxos) 중앙값 사용을 동기 부여했습니다. 일곱 및 아홉 복제본의 EPaxos는 요청 시간 초과도 보였으며, 이는 Section 4의 페루프 동시성 회계를 통해 그 크기에서 감소의 일부를 설명합니다.

- 클라이언트 동시성은 좁히지 않고 구현 간 격차를 넓혔습니다: EPaxos 이점은 테스트된 가장 높은 동시성에서 가장 컸고, 가장 낮은 동시성에서는 두 구현의 지연 시간 순서가 뒤집혔습니다.
- 쓰기 비율과 충돌 비율은 체계적인 처리량 추세를 만들지 않았습니다(Kruskal-Wallis $H \leq 4.44$, 임계값 9.49 및 12.59 대비). 충돌 검증은 구성된 비율이 워크로드에서 정확히 실현되었고 네 구현 중 세 개가 여전히 응답하지 않았으며 nvb EPaxos는 단조롭게 감소했음을 보여주었습니다: 충돌 민감도는 구현별입니다.
- Raft 리더의 장애는 Raft 팔로워나 EPaxos 복제본의 장애보다 두 자릿수 더 큰 가용성 공백을 만들었습니다.
- 평가된 HashiCorp Raft 구현에서 선출된 리더는 CPU 핫스팟이었으며, 세 복제본에서 팔로워 CPU의 2.2배, 아홉에서 3.8배를 사용했고, 두 프로토콜 패밀리는 전반적으로 유사한 복제본별 CPU를 사용했습니다.
- 구현 선택이 측정된 관계를 바꿉니다. 유지된 처리량의 분포는 두 지연 메커니즘 사이에 약 20배 차이가 납니다(9.0–18.1 % 대 0.7–28.4 %); 저장 모드를 일치시키면 테스트된 두 복제본 수 모두에서 기록된 순서가 뒤집힙니다; 세 복제본에서 기록된 EPaxos 이점은 각 구현이 실행된 저장 모드를 반영합니다.

5 논의

5.1 RQ1: 워크로드에 걸친 처리량 및 지연 시간

RQ1은 평가된 리더 기반 및 리더 없는 구현의 측정된 처리량과 지연 시간이 복제본 수, 클라이언트 동시성, 충돌률, 읽기/쓰기 구성에 따라 어떻게 변하는지 묻습니다. 가장 명확한 차이는 복제본 수와 클라이언트 동시성에서 관찰된 반면, 테스트된 충돌 및 쓰기 비율은 비교적 적은 체계적인 변동을 만들었습니다.

복제본 수를 늘리면 두 구현 모두에서 처리량이 감소했지만, EPaxos/Raft 비율은 일정하게 유지되지 않고 클러스터 크기에 따라 바뀌었으며, 차이는 세 및 다섯 복제본 구성에서만 통계적으로 감지 가능했습니다(Section 4, Table 3). 일곱 및 아홉 복제본의 EPaxos 감소는 프로토콜 확장의 속성이 아니라 클라이언트 시간 초과와 산물입니다: 각 시간 초과 요청은 전체 클라이언트 시간 초과 동안 클라이언트 워커를 점유하며, 해당 항을 포함한 페루프 회계는 그 크기의 모든 실행과 일치합니다. 따라서 그 크기의 낮은 EPaxos 처리량은 프로토콜의 확장 특성으로 읽어서는 안 됩니다.

클라이언트 동시성은 다른 패턴을 만들었습니다: EPaxos는 높은 동시성에서 Raft보다 훨씬 더 많이 얻었고, 격차는 테스트된 가장 높은 동시성에서 가장 컸습니다. 이는 평가된 EPaxos 구현이 여러 복제본을 통해 제안을 수락하는 반면 평가된 Raft 요청 경로는 선출된 리더에 집중되는 것과 일치하며, Raft 전용 후속 실험은 복제본 확장 패널티가 더 높은 동시성에서 지속됨을 확인했습니다(c32-c256에서 아홉 복제본 43–51 % 낮음) 리더의 CPU 사용률이 동시성에 따라 증가하면서 리더에 집중된 상태를 유지했습니다. 그러나 실험은 그 프로토콜 메커니즘을 구현 수준의 요청 처리, 스케줄링, 전송 비용에서 분리하지 않으며, 리더는 두 코어 할당에 접근했지만 초과하지 않았으므로 측정은 하드 CPU 용량 한계를 입증하지 않고 리더 중심 쓰기 경로와 일치합니다.

지연 시간 결과는 동일한 조건 의존성을 보여줍니다. Raft의 p50 지연 시간은 복제본 수에 따라 단조롭게 증가한 반면 EPaxos p50은 좁은 범위에 머물렀고, 테스트된 가장 낮은 동시성에서는

두 구현의 순서가 뒤집혔습니다. 따라서 측정된 지연 시간 관계는 어느 설계의 고정된 속성이 아니라 운영 조건에 의존합니다.

쓰기 비율과 충돌 비율 스윙은 평가된 구현에 대해 통계적으로 유의한 처리량 추세를 만들지 않았으며, 충돌 검증 패밀리는 이것이 워크로드가 구성된 비율을 실현하지 못한 산물이 아님을 보여줍니다: 실현된 핫 키 접근 패턴은 구성과 정확히 일치했고, 키 스페이스가 냉기를 유지하는 범위에서 네 구현 중 세 개가 여전히 체계적인 응답을 보이지 않았습니다. 네 번째인 nvb EPaxos는 구성된 비율에 따라 단조롭게 감소하여 충돌 민감도를 프로토콜 패밀리의 속성이 아닌 구현별로 만듭니다.

이 결과는 이전 비교와 관련이 있습니다. Howard와 Mortier는 구현과 구성이 프로토콜 패밀리보다 더 중요하다는 것을 발견했으며 [3], 이는 우리가 같은 패밀리의 구현들 사이에서 측정한 분포와 일치합니다. van Renesse 등과 Tollman 등은 충돌이 두 설계의 상대적 성능을 바꾸고 지리 복제에서 꼬리를 악화시킨다고 보고했습니다 [7, 9]; 여기서 구성된 비율은 네 구현 중 세 개에서 중앙값뿐 아니라 꼬리도 변화시키지 않았고(0 % 대비 75 %에서 p99 비율 0.96–1.09) nvb EPaxos의 중앙값과 꼬리를 함께 움직였습니다. 교차 사이트 또는 동일 키 경합이 필요한 메커니즘은 우리가 측정할 수 있는 구성인 단일 호스트 핫 키 워크로드에는 나타나지 않습니다.

전반적으로 RQ1은 복제본 수와 동시성이 측정된 성능에서 가장 눈에 띄는 변화를 만들었고, 쓰기 비율과 충돌 비율은 더 약하거나 구현별 효과를 만들었음을 나타냅니다. 이러한 발견은 기저 합의 프로토콜에 해당 요인이 중요하지 않다는 것을 확립하기보다 평가된 구현과 워크로드를 특성화합니다.

5.2 RQ2: 장애 및 통신 동작

RQ2는 복제본 장애와 증가하는 통신 비용 하에서 평가된 구현의 동작을 조사합니다. Raft 리더의 장애와 비리더 복제본의 장애는 테스트된 구성에서 명확히 다른 결과를 가집니다: 리더 종료는 대체자가 선출될 때까지 쓰기를 중단시킨 반면, Raft 팔로워나 EPaxos 복제본의 종료는 약 두 자릿수 더 짧은 중단을 만들었습니다(Table 4).

이는 평가된 Raft 구현에서 선출된 리더의 역할과 일치하지만, 테스트된 장애 메커니즘과 구현의 속성으로 읽어야 하며 리더 기반과 리더 없는 합의 사이의 보편적 구분으로 읽어서는 안 됩니다. 선거 실험은 복구가 중단된 선거 수와 관련이 있음을 보여주지만, 총 가용성 공백과의 연관성은 부분적으로 기계적입니다. 왜냐하면 부과된 격리 간격이 그 공백에 직접 기여하기 때문입니다; 격리 간격을 제거하면 남은 복구 간격과의 잔여 연관성은 약하며 기술적(descriptive)으로 취급되고 인과적이지 않습니다.

통신 실험은 두 번째 형태의 민감도를 드러냅니다. 애플리케이션 수준 주입에서 네 구현은 급격히 다른 것처럼 보였지만, tc/netem으로 구현 외부에서 동일한 명목 지연을 적용하면 유지된 처리량의 분포가 약 40배에서 2배로 압축되었고 구현 순서도 바뀌었습니다. 따라서 애플리케이션 수준 조건의 극단적인 길보기 민감도는 추가 지연 자체가 아니라 그 메커니즘이 각 구현의 전송 경로와 상호작용하는 방식에서 비롯됩니다: 메시지별로 대기하기 때문에 요청당 더 많은 메시지를 보내거나 전송을 직렬화하는 구현이 더 강하게 영향을 받습니다. 통제된 조건이 여기서 지연 민감도의 증거로 사용됩니다; 애플리케이션 수준 측정은 각 구현의 전송 경로를 특성화합니다. 통신 비용은 통제된 조건에서 지연 시간에도 영향을 미치지만 애플리케이션 수준 조건이 시사한 것보다 구현 간에 덜 발산적으로 영향을 미칩니다. 지터 스윙은 같은 해석을 지원하지만, 구현된 지터 모델이 평균 주입

지연과 분산을 모두 올리므로 그 실험은 둘을 분리할 수 없다는 한계가 있습니다.

종합하면 장애와 통신 실험은 불리한 조건 동작이 실패한 복제본의 역할과 네트워크 비용을 부과하는 통신 메커니즘 모두에 의존하며, 내부 메시지 처리 경로 외부에서 동일한 지연이 도입되면 통신 패널티가 평가된 구현 전반에 걸쳐 비슷한 크기임을 보여줍니다.

5.3 RQ3: 성능 차이의 구현 수준 원인

RQ3은 Raft와 EPaxos 사이의 측정된 성능 차이 중 얼마나 많은 부분이 합의 알고리즘 패밀리 자체가 아니라 구체적인 구현에 귀속되는지 묻습니다. 구현, 영속성, 통신 실험의 결과는 측정된 관계가 구현 세부 사항에 상당히 의존함을 일관되게 나타냅니다.

통제된 통신 지연 패밀리는 통신 비용에 대한 구현 민감도를 비교하는 주요 근거이며, 그 기준선 순서는 리더 기반 대 리더 없는 분류와 일치하지 않습니다: 0 지연 처리량이 가장 높은 구현이 지연이 적용된 후 가장 많은 처리량을 유지하는 구현이 아닙니다 (Section 4.8).

영속성 일치 패밀리는 특히 직접적인 증거를 제공합니다. 기록된 기준선은 영속 HashiCorp Raft와 메모리 내 원본 EPaxos를 비교하며, 저장 모드를 일치시키면 테스트된 두 복제본 수 모두에서 순서가 뒤집히고 모든 일치된 비교가 $p < 0.001$ 에서 유의합니다 (Section 4.7, Table 6). 영속성 비용 자체는 균일하지 않고 구현별이며, 로그가 영속화될 때 메모리 내 처리량의 가장 큰 비율을 잃는 구현이 기록된 비교에서 메모리 내 EPaxos와 대치된 구현입니다. 따라서 기록된 기준선은 합의 동작과 영속성 구성의 상당한 차이를 결합하며, 그 EPaxos/Raft 비율은 두 프로토콜 패밀리의 내재적 속성으로 취급될 수 없습니다.

자원 측정에는 두 번째 관찰을 추가합니다. 평균 복제본별 CPU 사용률은 처리량이 다른데도 두 기준선 구현 사이에서 유사했고, Raft 확장 실행에서 선출된 리더는 팔로워 CPU의 2.2에서 3.8배를 소비했으며 비율은 클러스터 크기에 따라 증가했습니다. 따라서 리더 중심 요청 경로가 암시하는 집중은 자원 사용에 나타납니다; 처리량 결과가 보여주지 않는 것은 그에 대한 깨끗한 확장 패널티인데, 가장 큰 크기의 EPaxos 측정이 클라이언트 시간 초과로 혼동되기 때문입니다 (Section 4).

결합된 결과는 Raft-EPaxos 비교의 구현 민감한 해석을 지원합니다: 측정된 처리량과 지연 시간은 합의 구조와 영속성, 요청 처리, 스케줄링, 메시지 처리, 전송, 워크로드, 장애 조건의 상호작용에서 발생하므로 평가된 구현 사이의 차이는 알고리즘 패밀리만으로 귀속될 수 없습니다. 이것이 알고리즘 구조가 무관하다는 것을 의미하지는 않습니다. 비대칭 장애 동작, 클라이언트 동시성에 대한 다른 응답, 복제본 수에 따른 다른 지연 시간 확장은 모두 각 합의 설계에서 발생하는 메커니즘과 일치합니다; 결과는 그러한 메커니즘이 구체적인 구현을 통해 관찰되며 구현 비용이 측정된 관계를 실질적으로 증폭, 약화, 또는 역전시킬 수 있음을 나타냅니다.

5.4 비교 해석의 함의

세 연구 질문은 함께 단일 구현 쌍과 단일 로컬 네트워크 워크로드에 기반한 비교가 여러 개별 효과를 혼동할 수 있음을 보여줍니다: 측정된 관계는 워크로드와 시스템 규모, 장애 역할과 통신 조건, 영속성 구성, 메시지 처리, 통신 비용 도입 메커니즘에 따라 변합니다. 통제된 네트워크 실험은 EPaxos와 Raft의 상대적 WAN 성능에 대한 포괄적 주장을 지원하지 않으면서 통신 민감도의 측정 가능한 차이를 입증하며, 장애 결과는 모든 리더

기반 시스템이 같은 비용을 부담한다는 것을 확립하지 않고 Raft 리더 교체 비용의 가용성 비용을 보여줍니다: 선거 시간 초과 구성, 전송 구현, 재시도 정책, 장애 감지가 모두 측정된 복구 간격에 영향을 줄 수 있으며, 같은 원칙이 테스트된 EPaxos 복제본 장애 시나리오에서 측정된 짧은 복구에도 적용됩니다.

따라서 실험 비교를 위해 결과는 프로토콜 효과를 시스템 수준 효과와 분리하려 할 때 영속성, 전송, 통신 조건, 워크로드, 장애 구성, 구현별 동작을 통제하거나 명시적으로 보고해야 함을 나타냅니다.

6 타당성 위협

6.1 외적 타당성

모든 복제본은 브리지 네트워크를 통해 단일 호스트에서 실행되므로 측정에는 광역 지연 시간, 실제 패킷 손실, 다중 머신 장애 도메인이 없습니다. 호스트는 12개의 물리 코어를 가진 WSL2 환경으로, 컨테이너와 하드웨어 사이에 가상화 계층이 있습니다. 클러스터 크기는 3-9, 키스페이스는 1,000개 키, 명령은 크기가 균일하며 측정 단계는 20s입니다. 따라서 결과는 긴 수명의 배포, 편향된 접근 패턴, 큰 페이로드, 지리적으로 분산된 복제본이 아닌 짧고 균일한 단일 호스트 실행을 설명합니다.

더 큰 클러스터 크기는 복제본 수 효과와 완전히 분리할 수 없는 물리적 자원 경합도 도입합니다. 각각 2.0 CPU를 요청하는 아홉 개의 복제본은 클라이언트와 마스터와 함께 12코어 호스트에 21개 CPU를 요청하며, 각 복제본은 2.0 CPU를 할당받는 동안 $GOMAXPROCS=4$ 이므로 Go 스케줄러 병렬성이 컨테이너 CPU 할당량을 초과합니다. 결과적으로 7개 및 9개 복제본 구성은 프로토콜 수준 확장의 고립된 증거가 아니라 자원 제약이 있는 로컬 환경에서의 측정으로 해석되어야 합니다. 이러한 구성은 이 환경에서 평가된 시스템의 동작을 나타내므로 유지되지만, 3개 및 5개 복제본 구성이 기본 확장 비교에 덜 자원 경합된 근거를 제공합니다.

읽기는 두 프로토콜 모두에서 합의를 통해 라우팅되며 Raft 특화 읽기 최적화가 활성화되지 않으므로 읽기 비교는 각 시스템의 가능한 가장 빠른 읽기 경로가 아닌 동등한 합의 지향 경로의 비교입니다. 두 시스템 모두 동일한 CPU와 메모리 할당으로 측정되어 통제된 비교를 제공하지만 자원 조정된 배포는 아닙니다.

영속성은 구현 비대칭의 원천입니다. 원래 기준선에서 평가된 Raft 구현은 영속 저장을 사용하는 반면 EPaxos 구현은 메모리 내 상태를 사용합니다. 특히 nvb EPaxos 구현은 내장 영속 저장 백엔드를 제공하지 않으며 이 연구를 위해 새 영속성 메커니즘이 구현되지 않습니다. 결과적으로 원래 지연 시간과 처리량 측정에는 합의의 프로토콜 자체보다 평가된 구현에 특정한 영속성 비용이 포함될 수 있습니다. 따라서 별도의 영속성 통제 실험이 각 구현이 두 모드를 지원하는 곳에서 메모리 및 영속 구성을 평가합니다; 그 결과는 Section 4.7에 보고되며 기록된 순서가 각 구현이 실행된 저장 모드에 의존함을 보여줍니다. nvb 구현은 지원되는 메모리 내 구성에서만 평가됩니다.

EPaxos 구현은 7개 및 9개 복제본 구성에 필요한 의존성 집합 크기를 지원하도록 추가로 수정됩니다. 그 크기의 결과는 프로토콜 동작과 필요한 구현 확장의 비용을 결합합니다.

통제된 네트워크 지연 실험은 애플리케이션 수준 대기 대신 운영 체제 수준 네트워크 에뮬레이션을 사용하여 복제본 간 트래픽에 고정 단방향 지연을 적용합니다. 지연은 피어 간 복제본 트래픽으로 범위가 제한되어 클라이언트-복제본 트래픽은 영향을 받지 않습니다. 이 접근 방식은 애플리케이션 수준 지연 주입의 어댑터 수준 직렬화 효과를 피하지만 패킷 손실, 혼잡, 다중 홉

라우팅, 독립 링크 장애를 포함한 분산 물리 네트워크의 전체 동작을 재현하지는 않습니다. 이전 애플리케이션 수준 지연 및 지터 측정은 레거시 실험으로 유지되지만 깨끗한 네트워크 지연 민감도의 주요 증거로 취급되지 않습니다(Section 4.8); 지터는 통제된 지연 실험에 포함되지 않습니다.

충돌 비율은 워크로드의 키 접근 패턴을 통해 지정되지만 구성된 백분율 자체는 EPaxos가 관찰한 프로토콜 수준 경합의 양을 확립하지 않습니다. 따라서 충돌 검증 실험은 구현이 노출하는 곳에서 빠른/느린 경로 동작을 포함한 사용 가능한 프로토콜 수준 실행 지표와 함께 실현된 핫 키 접근 패턴을 기록합니다(Section 4.9). 충돌 민감도에 대한 결론은 워크로드 구성과 이러한 관찰된 실행 특성 모두가 지원하는 효과로 제한됩니다.

장애 및 선거 실험은 세 개의 복제본을 사용하므로 그 복구 시간은 해당 구성만 특성화합니다. 장애 시나리오도 대칭적이지 않습니다: Raft 리더 장애 실험은 정상 요청 엔드포인트를 제거하는 반면, 팔로워 및 복제본 장애 실험은 비중앙 복제본을 제거합니다. Raft 리더 복구 간격은 구성된 하트비트 간격과 선거 시간 초과, 클라이언트 시간 초과 및 재연결 동작에 의존합니다. 따라서 이러한 매개변수는 명시적으로 보고되며 측정된 가용성 공백은 Raft의 구현 독립적 복구 시간이 아닌 지정된 장애 감지 구성 하의 결과로 해석됩니다.

정확성 하네스는 테스트된 워크로드와 재시작 시나리오 하에서 상태 수렴을 검증하지만 합의의 정확성의 공식 검증을 구성하지 않으며 일반적인 선형화 가능성을 확립하지 않습니다. 특히 테스트된 워크로드는 순서 위반을 노출할 수 있는 충돌 연산과 메시지 재정렬의 모든 가능한 패턴을 실행하지 않습니다. 하네스는 요청당 고유 키를 쓰므로 같은 키에 대한 동시 쓰기를 실행하지 않으며 기록된 이력에 대해 선형화 가능성 검사기가 실행되지 않았습니다; 따라서 경합 하에서 연산 순서 의미론의 독립적인 테스트가 아닌 수렴 검사입니다.

6.2 통계적 결론 타당성

각 구성은 열 번 측정되며 집계는 비모수 부트스트랩 95% 신뢰 구간이 있는 중앙값으로 보고됩니다. 열 번의 반복은 상당한 불확실성을 남깁니다: 네 개의 정상 상태 패밀리에서 동일한 명목 구성을 측정하면 두 프로토콜 모두에서 2-3% 차이가 나는 중앙값이 생성되었고, 부트스트랩 구간은 일곱 및 아홉 복제본과 같이 중앙값이 다른 여러 비교에서 겹칩니다. 따라서 우리는 겹치는 비교를 작지만 확립된 효과의 증거가 아닌 결정적이지 않은 것으로 취급합니다.

스위트는 많은 구성과 쌍별 비교를 포함하며 다중성 보정이 적용되지 않습니다(Section 3): 평가에서 인용된 p -값은 해당 단일 비교의 보정되지 않은 값입니다. 검정은 속한 패밀리 옆에 보고되므로 주장 뒤의 비교 수가 보이며, 우리는 경계선 유의성보다 잘 분리된 비교에 의존합니다. 분리된 보정 선택이 중요하지 않을 만큼 넓습니다: 유의한 것으로 보고된 모든 결과는 $p \leq 0.001$ 이며, 여기 어떤 패밀리도 열 개 이상의 검정을 포함하지 않으므로 $\alpha = 0.05$ 의 Holm-Bonferroni 보정은 각 결과를 유의하게 남깁니다.

선거 상관 분석은 특별한 주의를 요구합니다: 측정된 실패 선거 수는 복구 과정의 결과이며 주입된 격리 기간과 기계적으로 관련되고, 60개 실행은 여섯 개의 격리 목표(각각 열 개 실행)로 그룹화됩니다. 따라서 관측은 60개의 완전히 독립적인 표본으로 취급되어서는 안 됩니다. 상관은 주로 기술적 연관성으로 해석되며 일반적인 인과 관계의 독립적 증거로 해석되지 않습니다.

20s 측정 창은 장기 정상 상태 동작의 특성화를 제한하며, 페루프 클라이언트는 구성된 동시성이 독립적으로 통제된 도착률이 아닌 인플라이트 요청의 최대 수임을 의미합니다. 깨끗한 실행 내의 이상값은 제거되지 않습니다; 환경 간섭의 영향을 받는 것으로 식별된 실행은 별도로 조사되며 성능이 비교에 불편하다는 이유만으로 제외되지 않습니다.

7 결론

본 연구는 Raft와 EPaxos를 대표적인 리더 기반 및 리더 없는 합의 설계로 통제된 실험 비교를 수행했으며, 복제본 수, 클라이언트 동시성, 워크로드 구성, 충돌 비율, 장애 모드, 통신 비용, 영속성 구성을 변화시켜 합의 구조와 관련된 효과를 구현 및 실험 환경에서 도입된 효과와 구분하고자 했습니다.

복제본 수를 늘리면 두 구현 모두에서 처리량이 감소했고, 평가된 EPaxos 구성은 높은 클라이언트 동시성에서 더 높은 처리량을 유지했습니다; Raft 특화 확장 실험은 이 패널티가 동시성이 증가함에 따라 지속되며 CPU 사용률이 리더에 점점 집중됨을 보여주었습니다. 이러한 관찰은 리더 중심 요청 경로와 일치하지만, 실험이 프로토콜 구조만으로 관찰된 차이를 설명한다는 것을 확립하지는 않습니다. 워크로드 구성은 균일한 응답을 만들지 않았습니다: nvb EPaxos만 구성된 충돌 비율에 응답했습니다.

장애 실험은 중앙 집중식 요청 처리의 또 다른 결과를 드러냅니다: 테스트된 세 복제본 구성에서 리더 장애는 대체 리더가 확립될 때까지 쓰기 처리를 중단시킨 반면, Raft의 팔로워 장애와 EPaxos의 복제본 장애는 상당히 더 짧은 중단을 만들었습니다. 이러한 결과는 프로토콜 독립적 복구 경계를 확립하기보다 평가된 시스템의 관찰된 장애 동작을 특성화합니다.

구현 민감도 실험은 직접적인 프로토콜 수준 비교에 중요한 한계를 제공합니다. 같은 합의 패밀리에 속한 구현들은 절대 처리량과 지연 시간에서 달랐고 통제된 통신 지연에 동일하게 응답하지 않았습니다; 구현 수준 메커니즘은 기저 합의 알고리즘을 바꾸지 않고 측정된 결과를 실질적으로 바꿀 수 있습니다. 따라서 절대 성능 측정은 Raft나 EPaxos의 내재적 성능 상수가 아니라 특정 구현과 환경의 속성으로 해석되어야 합니다.

종합하면 발견은 리더 기반과 리더 없는 합의 사이의 보편적 성능 순서를 지지하지 않습니다: 측정된 관계는 이러한 요인의 상호작용에서 발생하며, 어느 관찰도 어느 프로토콜 패밀리에 대한 일반적 우월성 주장으로 해석되어서는 안 됩니다.

본 연구는 또한 경험적 합의 연구에 대한 방법론적 고려 사항을 제시합니다: 영속성 일치와 통신 비용 주입을 포함한 통제된 후속 실험은 관찰된 차이가 주변 구현 환경의 변화에 견고한지 결정하는 유용한 증거를 제공하며, 성능은 단일 처리량 또는 지연 시간 수치로 축소되기보다 통제된 시스템 조건의 함수로 분석될 때 가장 유익합니다.

이러한 발견의 일반화를 제한하는 몇 가지 한계가 있습니다: 실험은 단일 호스트 컨테이너 환경에서 수행되었고 운영 체제 수준 네트워크 에뮬레이션을 사용했으며, 제한된 범위의 클러스터 크기와 페루프 워크로드를 사용했고, 정확성 실험은 선형화 가능성보다 수렴을 검증했습니다.

향후 작업은 방법론을 다중 호스트 및 지리적으로 분산된 환경, 추가 독립 구현, 더 큰 복제본 집합, 더 강한 경합과 더 다양한 접근 패턴을 가진 워크로드로 확장할 수 있습니다. 저장, 전송, 스케줄링, 애플리케이션 계층의 프로파일링은 프로토콜 수준 동작을 구현 오버헤드와 더 분리할 수 있습니다.

감사의 글

생성형 AI 도구가 이 작업의 준비 과정에서 벤치마크 및 랩 코드 개발, 언어 편집, 교정, 원고 텍스트의 초안 작성 및 수정에 사용되었습니다. 저자는 연구 질문, 실험 설계, 전반적인 방법론, 연구 해석을 독립적으로 정의했습니다. 실험 실행, 데이터 분석, 보고된 결과의 검증은 저자가 수행하고 확인했습니다. AI 생성 제안은 저자가 검토하고 수정했으며, 저자는 원고의 최종 내용과 기술적 주장에 책임이 있습니다.

참고 문헌

- [1] etcd Authors. 2025. etcd: Distributed Reliable Key-Value Store (raft package). <https://github.com/etcd-io/etcd/tree/main/raft>. Version v3.7.0.
- [2] HashiCorp. 2025. HashiCorp Raft: A Raft Consensus Protocol Implementation in Go. <https://github.com/hashicorp/raft>. Version v1.7.3.
- [3] Heidi Howard and Richard Mortier. 2020. Paxos vs Raft: Have We Reached Consensus on Distributed Consensus?. In *7th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC 20)*. ACM, 1–9. doi:10.1145/3380787.3393681
- [4] Leslie Lamport. 1998. The Part-Time Parliament. *ACM Transactions on Computer Systems* 16, 2 (1998), 133–169. doi:10.1145/279227.279229
- [5] Iulian Moraru, David G. Andersen, and Michael Kaminsky. 2013. There Is More Consensus in Egalitarian Parliaments. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP 13)*. ACM, Farmington, PA, 358–372. doi:10.1145/2517349.2522738
- [6] Diego Ongaro and John Ousterhout. 2014. In Search of an Understandable Consensus Algorithm. In *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC 14)*. USENIX Association, Philadelphia, PA, 305–319. <https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>
- [7] Sarah Tollman, Seo Jin Park, and John Ousterhout. 2021. EPaxos Revisited. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*. USENIX Association, 613–632. <https://www.usenix.org/conference/nsdi21/presentation/tollman>
- [8] Nathan van Benschoten. 2025. EPaxos: Egalitarian Paxos in Go. <https://github.com/nvanbenschoten/epaxos>. Commit 425bd36.
- [9] Robbert van Renesse, Nicolas Schiper, and Fred B. Schneider. 2015. Vive La Différence: Paxos vs. Leaderless Replication. In *Proceedings of the 28th ACM Symposium on Principles of Distributed Computing (PODC 15)*. ACM, 242–251. doi:10.1145/2767386.2767419